International Journal of Software Engineering and Knowledge Engineering
© World Scientific Publishing Company

# MODERNIZING 90S ERA SOFTWARE TO A NEW LANGUAGE AND ENVIRONMENT USING LLMS - AN EMPIRICAL INVESTIGATION

DRAGAN BOJIĆ

*School of Electrical Engineering, University of Belgrade, Bulevar kralja Aleksandra 73*
*Belgrade, 11120, Serbia*
*bojic@etf.bg.ac.rs*

DRAŽEN DRAŠKOVIĆ

*School of Electrical Engineering, University of Belgrade, Bulevar kralja Aleksandra 73*
*Belgrade, 11120, Serbia*
*drazen.draskovic@etf.bg.ac.rs*

Received (3 September 2024)
Revised (28 April 2025)
Accepted (30 April 2025)

Legacy software, particularly from the 1990s, often becomes obsolete due to aging hardware and outdated software environments. Traditionally, software modernization required extensive manual effort, involving reverse engineering, code rewriting, and re-architecting. However, advancements in large language models (LLMs) have introduced new possibilities for automating software translation and modernization. This paper explores the feasibility of using LLMs for modernizing 90s-era Windows applications, specifically migrating legacy C and C++ code to Python. Our methodology includes decompilation, source code analysis, automated translation using ChatGPT, and user interface reconstruction. We empirically evaluate three software projects by analyzing LLM-based translation accuracy across different code structures, including algorithmic logic, file handling, and graphical interfaces. Results indicate that while LLMs achieve high translation accuracy (~88%) for structured code, challenges persist in handling decompiled code and user interface generation. The study provides insights into the effectiveness and limitations of LLMs in real-world software renovation, offering guidelines for leveraging machine learning in legacy system modernization. These findings contribute to both academic research and practical applications, suggesting a pathway for cost-effective and scalable legacy software migration.

*Keywords*: decompilation, software translation, machine learning.

## 1. Introduction

Software aging is defined as the inability to further adapt to evolving user needs or changes in the software/hardware environment [1]. During this gradual process of aging, software is at some point marked as outdated, based on outdated technologies and standards. There is a growing repository of legacy software, The Internet Archive Software Collection [2], for example, contains over a million vintage software titles. We often have motives to keep legacy software alive in the sense that it can still be executed and used. The most common approach is to create an emulator that replicates the original environment, allowing the unmodified binary to run. Apart from various gaming console

emulators, there are also emulators of pre-relational database environments from the 1960s [3], adapted for modern operating systems such as Unix and Windows.

Another approach to preserving legacy software involves migration through re-architecting (e.g., transitioning from a monolithic architecture to microservices) or rewriting/rebuilding it for modern hardware and software environments. However, this strategy comes with a series of warnings about risks and costs, with a recommendation to give priority to all alternative strategies (retirement, leaving the original environment, rehosting to another environment without modifications, acquiring a functionally equivalent new product, etc.). The traditional method of rewriting legacy software involved documenting its functionality and user requirements through interviews with users and domain experts - sometimes formally modeling them - before reimplementing the system from scratch [4]. In general, this was seen as a difficult problem, because many business concerns are deeply buried within the code and often over time the structure of the software deviated significantly from the initial one and the technical documentation became out of date. An example that received publicity is described in [5] where Microsoft was forced to patch bugs in the Equation Editor binary, a component of the Office suite, because apparently the source code was lost over time.

With the advancement of machine learning (ML) models based on large language models (LLM) that can effectively assist in software development, new opportunities for legacy software modernization are emerging.

## 2. Existing solutions

Automated code translation has been a long-standing research challenge in software engineering, driven by the need to migrate legacy software to modern platforms. Early efforts relied on rule-based transpilers, which required substantial manual effort and domain expertise to encode language-specific transformation rules. More recently, neural machine translation (NMT) approaches and large language models (LLMs) have been explored for source-to-source code translation, demonstrating significant improvements over traditional methods [6][7].

Conventional approaches to legacy software migration primarily depended on manually developed rule-based transpilers or semi-automated tools. Notable examples include C2Rust for C-to-Rust translation and tools designed to convert legacy COBOL code to Java [8][9]. However, these rule-based approaches were costly and time-consuming, as each language pair required extensive rule engineering. Furthermore, they struggled with non-trivial code structures, such as complex memory management patterns and API differences, resulting in incomplete or incorrect translations.

In recent years, neural network-based methods, particularly those leveraging sequence-to-sequence models, have gained prominence in code translation. Studies such as TransCoder [10] demonstrated that unsupervised learning could improve translation accuracy by training models on large-scale monolingual code datasets. Variants like TransCoder-IR and TransCoder-ST further incorporated intermediate representations and automated test case filtering to enhance correctness [11][7]. Despite these advances, these models required substantial computational resources for training and still exhibited failure rates exceeding 50% in many language translation settings.

The advent of LLMs, such as GPT-3.5, Codex, and LLaMA, has introduced a paradigm shift in code translation. Unlike task-specific models, LLMs are pre-trained on vast corpora of human-written code and natural language, enabling them to generalize across multiple code-related tasks, including translation, code generation, and program repair. A recent study [12] examined the effectiveness of LLMs for automated code translation, finding that models like GPT-3.5 achieved significantly higher computational accuracy, over 87% in some cases, compared to traditional learning-based transpilers. However, challenges remain, particularly in preserving program semantics and addressing syntax mismatches between source and target languages.

Empirical investigations into LLM-based code migration have revealed key challenges in their adoption [12]. Studies indicate that the primary sources of translation errors include: *Logic Failures* (38.51%), where LLMs fail to fully comprehend the underlying logic of the original code, leading to semantic inconsistencies; *Syntax and API Misuse* (41.38%), as variations in standard libraries and function signatures across languages often result in misinterpreted API calls and syntax structures; and *I/O Type Inconsistencies* (14.94%), where LLMs struggle with explicit type declarations, particularly when translating from dynamically typed languages like Python to statically typed ones like C++.

Existing experimental studies of the effectiveness of LLMs in language translation tasks frequently rely on multilingual datasets such as [13], which have serious limitations: the limited number of test cases (e.g., only 467 for C++ in dataset [13]) take isolated code fragments from different projects, the size of functions (e.g., for dataset [13], an average of 105.8 tokens) is often smaller than real functions, and they do not consider the impact of using specific language libraries in the source and target languages on translation performance. There are no universally accepted metrics for evaluating translation performance; standard data science metrics such as BLEU have been deemed inappropriate [12]. Moreover, there is no investigation of the influence of exact phrasing of queries to LLMs to the performance of translation.

While LLM-based code translation research has primarily focused on mainstream languages such as Python, Java, and C++, fewer studies have addressed the migration of 1990s-era Windows applications written in legacy Win16, Win32, OWL, or MFC frameworks. These applications often contain low-level memory management techniques, extensive use of GUI libraries, and file system operations that differ significantly from modern programming paradigms. An empirical study on using LLMs to migrate such applications remains unexplored, making our research a novel contribution to the field.

Our work builds on prior research in automated code translation and LLM-driven software migration but focuses specifically on the empirical evaluation of legacy Windows desktop software modernization. By quantifying the correctness of LLM-generated code, classifying translation accuracy by code structure (e.g., object-oriented vs. functional), and assessing post-translation manual modification rates, we aim to provide a deeper understanding of the feasibility and limitations of LLMs in real-world software renovation.

## 3. Problem statement

The task is to create a functionally identical or similar application for a modern environment on the basis of an existing software application, built for an outdated software environment. A legacy environment is characterized by hardware and software

(operating system, libraries, and development tools) that are no longer used. A modernized application is implemented in a different programming language and development environment that follows modern requirements, practices and conventions (for example in user interaction).

## 4. The proposed solution

There are some preconditions to be fulfilled considering the old software in order to proceed with its renovation: An old system must at least be executable in the original legacy environment (e.g. virtual machine with an old operating system and optionally old software development tools). The system should have functional specification, black-box tests (usually end-to-end system tests) and design documents. The optional (big) plus is that there is source code of the original system and that it can be rebuilt into executable version in the original environment. The automated unit tests could also help the process of verifying a new code.

The renovation methodology we adhered to this research consists of the following steps:

1. Choosing an interesting part of the old software system to renovate.
2. Optional phase: decompiling from binary code to old source code.
3. Analyze old source code to identify the part that implements the function of interest identifies in step 1. The analysis should also decompose it into back-end algorithmic part and front-end user interface part, as well as any other category of code (such as input/output or database access) that has some specific structure (e.g. using a standard library or standard design pattern) or function.
4. Translate the old algorithmic code to new language and environment using LLMs. Then test and correct the new code using the original tests adapted to the new environment.
5. Optional: Generate new front-end code using LLMs' generative features. Use the original functional specification and screenshots of the original user interface elements to form the query to LLM.
6. Integrate new algorithmic and user-interface code and conduct end-to-end tests using the original black-box tests that came with the legacy application.

We shall now discuss these steps in more detail. The motivation to choose the whole system or some interesting part is up to the person who conducts or finances the process. For our study, we choose some specific use case scenario of each of the systems studied: it is either some specific algorithm embedded in a system, or some specific visual representation of data, etc.

As for step 2, one of the investigated systems did not contain the original source code, just the binary code with the complete debug information included. The decompilation process from binary code to (presumed) source code in C was conducted using classical (not LLM-supported) decompilation tools. More details on this particular project are given in the next section. In the context of this research, we focused on the analysis of the

quality of the code obtained by decompilation and its suitability for translation into Python by means of the LLM model.

Step 3 deals with slicing the original source code to logically extract the part of the system functions declared interesting in step 1. Although code analysis with the aim of extracting parts of interest can be done by statically analyzing the code, to look for parts with standard patterns such as the use of some standard library or a framework, a very effective technique is the possibility of inserting "logical probes" in the form of log prints or message boxes into the original code in order to determine which parts of the code are actually executed within the defined system usage scenario. This requires that the original code be rebuildable in the original environment, usually using the original development tool on a virtual machine. After slicing the system to extract parts of interest, we performed manual code classification in several categories, as a basis for our statistical investigation, to be further discussed in Statistical Analysis section.

Step 4 is a quintessence of our study. The LLM used to conduct the full study was ChatGPT, free version 3.5 for Project A and paid 4o for Project B and Project C. We conducted a quick survey on parts of code on several other LLMs, such as Copilot, DeepSeek etc. An interesting detail is that DeepSeek recommends ChatGPT 4 as the best LLM for the task of translating legacy C++ code into Python. The conclusion was that all these LLMs could be effectively used to do the job, but we opted for ChatGPT based on practical considerations on availability and affordability, and a flexible way to interact with it: it can process up to ten file attachments in a chat.

Transpilation results are greatly influenced by the way the query is set. The most straightforward way, attaching the .cpp module of the class, as well as the included header files, does not always produce the best quality results. The most problematic are big very big C/C++ functions (500+ lines of code) and of high control flow complexity, with several levels of nested ifs and switches etc., which is not unusual in the old C code dealing with Windows API, even in C++ code before standard GoF design templates became widely known. The LLM model usually produces partial results, missing cases in switch statements, or missing deeply nested layers of control flow constructs.

ChatGPT sometimes automatically tries to change the structure of the original code. The most effective method we came up with by trying it is to set the context of the class, that is, a collection of .h files excluding those belonging to a standard framework such as MFC, and to set the following query:

*Please help me translate all methods from C++ to Python. Please preserve all details and logic from Microsoft C++ code as much as possible. The attached .h files provide declarations and context for translation. I will provide you with functions to translate, one by one.*

Then in the interactive session, methods of the original code are passed one by one. The quality and accuracy of the generated code is improved if the Python library to be used is explicitly specified, for example, wxPython for GUI code or pyodbc for project code where connection to a relational database is present in a code. Several observed

flaws in the generated code, which require manual refinement, are discussed in the chapter on statistical results.

| Generated ChatGPT code for Python | Generated code for “Python with pyodbc” |
|---|---|
| def update_to_rs(self, p_rs):<br>p_rs['DB_KO_BP_MIN'] = int(self.m_minBrProblema)<br>p_rs['DB_KO_BP_OPT'] = int(self.m_optBrProblema)<br>p_rs['DB_KO_BP_MAX'] = int(self.m_maxBrProblema)<br>p_rs['DB_KO_KVP_ID'] = int(self.m_kvpId)<br>p_rs['DB_KO_OBLAST_ID'] = int(self.m_oblastId)<br># throws TypeError: 'pyodbc.Cursor' object does not<br># support item assignment | def update_to_rs(self, cursor):<br>sql_update = "UPDATE KriterijumOblasti SET BpMin = ?, BpOpt = ?, BpMax = ?, Ponder = ? “<br>“, OblastId = ?, KriterijumVrsteProblemaId = ? WHERE KriterijumOblastiId = ? "<br>params = (<br>int(self.m_minBrProblema),int(self.m_optBrProblema), int(self.m_maxBrProblema),<br>int(self.m_ponder), int(self.m_kvpId), self.m_dbId )<br>cursor.execute(sql_update, params)<br>cursor.commit() # Save changes to the database |

Transpilation is not the only way to obtain new code. In Step 5 of our methodology, we mentioned a possibility to generate code using an LLM from the description of existing application look and behavior. In the project where no source code was available at all, this step seemed more efficient for GUI part of the application (dealing with graph editing, etc.) that decompiling the original binary code. Also, if you don't want to copy the behavior of the original application exactly, but you want more significant changes, for example in the appearance and behavior of the GUI, generating the code from the description seems like a viable solution. Our experiments with code generation by ChatGPT, and results of this approach are further discussed in the next section.

Finally, the new code must be verified. All selected projects had a set of system tests, and some also had automated tests but for legacy testing tools. In general, testing starts with the modules that read the input data. In the case of text files or a relational database, it is relatively easy to adopt new code to use such inputs. The biggest problem is code that reads and writes binary files, and in such cases ChatGPT did not immediately generate overly usable code considering the semantic differences between Python and C++. With a special query, ChatGPT could define formats for reading specific C records from a binary file that were then used to write code that converts the binary file into the pickle format native to Python. The converted files were used in code testing. In general, the phase of testing and debugging the integrated code was immediately followed (as in classic “big bang” integration), which was possible because the transpiled code was correct to a high percentage (as discussed in a Statistical Analysis section).

### *An illustrative example*

The procedure will be explained on the example of our tool for testing software [14] using the cause-and-effect graph method [15] written in the C programming language for Windows 3.1. The TestGraph application from 1997 has a GUI, shown in Figure 1, that allows creating and editing a cause-effect graph (abbr. CEG). Based on the graph, a

calculation of the decision table is performed by the tool. In addition to basic functions, the application allows creating and printing descriptions of manual tests based on the decision table, as well as working with files, recording and loading graphs and tests, and getting interactive help. The application was of dual purpose, as a commercial program where it did not experience significant success, and for educational purposes in academic studies. Over time, only the 16-bit executable binary code in Microsoft's New Executable (NE) format along with debug information remained of the application, as well as extensive technical documentation, which includes a description of each function, and a description of all data structures used. The application can run on Windows 2000, which is the last Windows that supported NE (16-bit) format. The most interesting part of the application is the Myers transformation algorithm from CEG to decision table. This old and well-known technique is mentioned in ISO standards [16], as well as many academic and professional certification programs. The creator of the technique did not provide a sufficiently precise specification or implementation of the algorithm, which gave rise to different interpretations [17]. Myers' idea is still relevant for research, e.g., combined with new ML approaches to automate parts that are manual in the original technique, i.e., pattern identification and consequences from specification and graph construction [18]. Our interpretation [14] follows Myers' original idea very faithfully and has been tested extensively. The starting motive for salvaging the old implementation is that it is more rational to use the old implementation, in which many months of algorithm design and testing were invested, than to write from scratch.

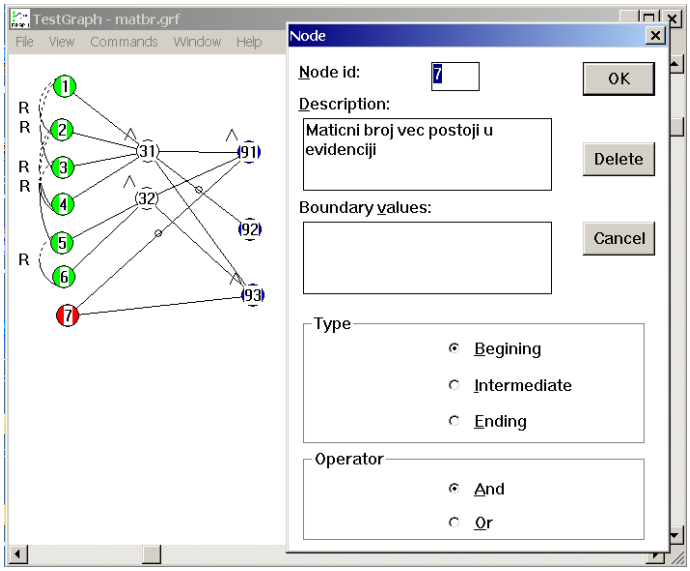
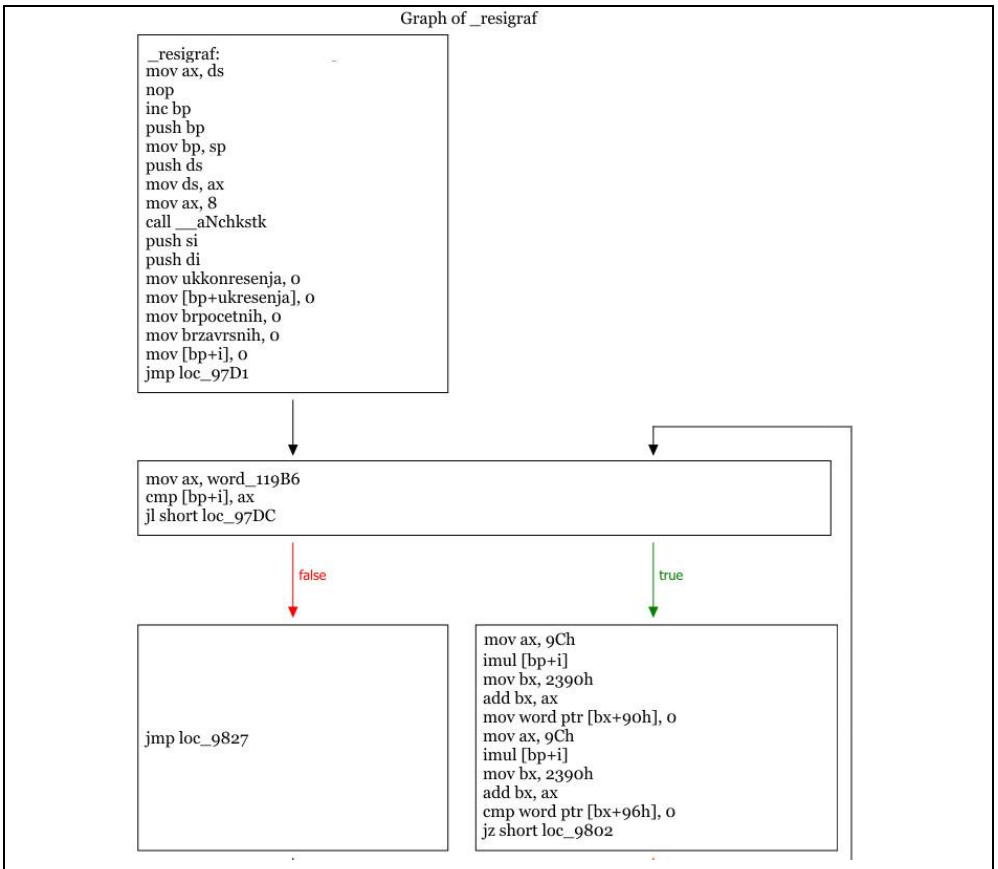



**Fig. 1. User interface of the original application TestGraph.**

In the first step, using the IDA Pro 7.0 tool, the existing application was disassembled, and the data was exported in xml format using the command

edit/plugins/XML exporter. Figure 2a shows a part of the disassembled call graph for a certain function of the original program, but this kind of information is of no use to us. The IDA Pro tool also has a decompiler, but it does not work for 16-bit x86 code. Therefore, the data was imported into the Ghidra tool, which has a 16-bit decompiler. Ghidra alone cannot extract all the information from binary code that IDA Pro can. Figure 2b depicts Ghidra decompiler output without IDA Pro information, most variable and function names are missing. but the information from IDA Pro can be combined to produce higher quality decompilation (Figure 2c). Note that function and variable names are in Serbian language.

In addition to information imported from IDA Pro, additional information was supplied to Ghidra to improve decompilation result. Declarations of data structures from the documentation that the Ghidra tool uses during decompilation were added manually in the form of imported .h file (with a command File/Parse C source…). Figure 2a shows a part of imported .h file. In order for the code to get a final look of the salvaged source code (Figure 2b), additional manual corrections were implemented: for loops that decompiler does not detect, and the proper indexing of arrays of structures (Figure 3).




(a)

```
void __cdecl16far FUN_1000_979c(void)
{
  int *piVar1;
  int unaff_BP;
  undefined2 local_a;
  int local_8;

  FUN_1000_dcdc(unaff_BP + 1);
  DAT_1008_22b8 = 0;
  local_a = 0;
  DAT_1008_2384 = 0;
  DAT_1008_665a = 0;
  local_8 = 0;
  while (local_8 < DAT_1008_1bf6) {
    *(undefined2 *)(local_8 * 0x9c + 0x2420) = 0;
    if (*(int *)(local_8 * 0x9c + 0x2426) == 0) {
      DAT_1008_2384 += 1;
    }
    else {
      if (*(int *)(local_8 * 0x9c + 0x2426) == 2) {
        DAT_1008_665a += 1;
      }
    }
    local_8 += 1;
  }
 local_8 = 0;
```

(b)

```
void __cdecl16far _resigraf(void)
{
  int *piVar1;
  Resenje *ukresenja_00;
  word ukresenja;
  word i;

  __aNchkstk();
  ukkonresenja = 0;
  ukresenja = 0;
  brpocetnih = 0;
  brzavrsnih = 0;
  i = 0;
  while ((int)i < header.numnode) {
    *(undefined2 *)((int)&node[0].brgrana+i*0x9c)= 0;
    if (*(int *)((int)&node[0].NodeType+i*0x9c)==0){
      brpocetnih += 1;
    }
    else {
      if(*(int *)((int)&node[0].NodeType+i*0x9c)==2){
        brzavrsnih += 1;
      }
    }
    i += 1;
  }
 i = 0;
```

c)

**Fig. 2. Three levels of reverse engineered code using IDA Pro and Ghidra.**

Only the code segments implementing the Myers algorithm and the save/load functionality for the CEG graph were regenerated. The statistical parameters of decompilation and subsequent translation into Python are reported in the next section for Project A. The correctness of the generated code was verified by testing it with graphs recorded from the original application, with the new application executed as a console-based program. The original documentation contains records of the testing procedures used for the legacy application, indicating that it underwent extensive unit and integration testing. However, the actual test cases themselves were not preserved.

```c
#define MAXSLOVA 70

typedef struct {
        int16_t x;
        int16_t y;
} Point;

typedef struct tnode {
        char    imecvora[MAXSLOVA];
        char    boundary[MAXSLOVA];
        char    op;
        char    id[3];
        int16_t brgrana;
        char    vred;
        char    _filler;
        bool    flag;
        int16_t NodeType;
        Point   position;
} Tnode;
```

(a)

```c
void resigraf(void)
{
  Resenje* ukresenja;
  int i;

  ukkonresenja = NULL;
  ukresenja = NULL;
  brpocetnih = 0;
  brzavrsnih = 0;
  for (i = 0; i < header.numnode; i++) {
    node[i].brgrana = 0;
    if (node[i].NodeType == 0) {
      brpocetnih += 1;
    }
    if (node[i].NodeType == 2) {
      brzavrsnih += 1;
    }
  }
```

(b)

**Fig. 3. (a) Structure declarations added manually.**
**(b) Final decompiled code after tidying up.**

In the next phase, the resulting C code was automatically transformed into equivalent Python code using ChatGPT 3.5. The output generated by ChatGPT was of high quality; the only difficulties were the limitation of the free version to 2048 characters in the input, which required splitting some functions. The translated Python code required certain manual adjustments in specific areas. These included handling parameters passed by reference in the original C code and correctly populating data structures from a binary file. The statistical impact of these modifications is presented in Table 3.

The Python code's correctness was verified using the same approach as for the salvaged C code. The verification process involved loading input data from graphs recorded in the original application and comparing the outputs of the new and original application. To illustrate the complexity of Myers' algorithm implementation, Figure 4 presents the function call tree from the original project documentation. The algorithm processes the CEG graph from the end nodes to the initial nodes. Node processing is recursively defined and combinatorial in nature. The return branches in the function call tree depict recursive functions.

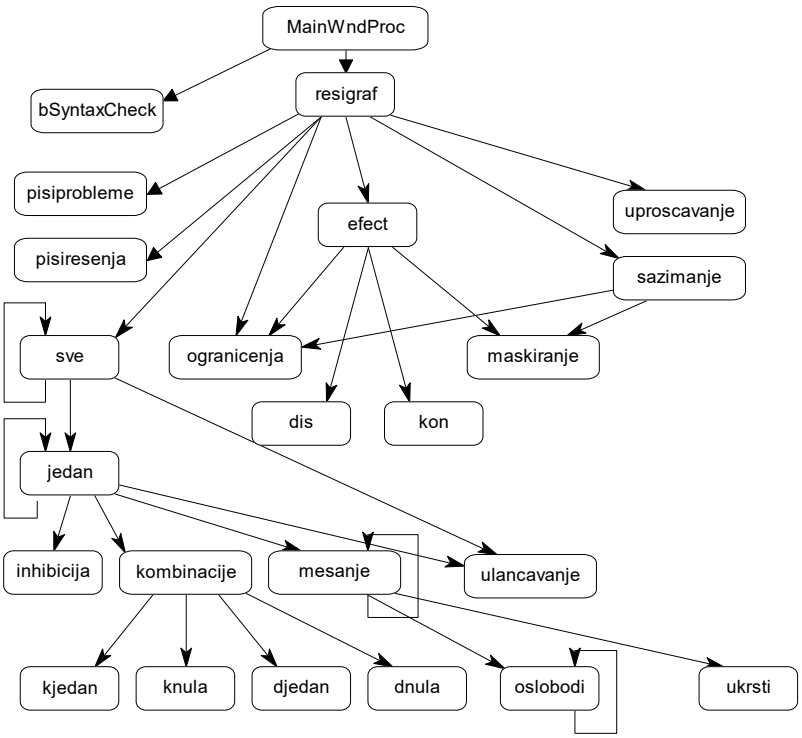



**Fig. 4. The function call tree for Myers' algorithm.**

In the next step of our salvaging process, the decision was made to implement the GUI for the application from scratch. It didn't make sense to decompile the GUI part of the application, for several reasons: (1) The original 16-bit Win32 API is relatively low-level and lacks a direct Python equivalent; (2) The original 1990s interface was outdated and needed improvement; (3) The new application was intended for educational purposes, requiring an additional visualization of the Myers algorithm's trace - an enhancement absent in the original, which only produced final results. The GUI library wxPython was chosen for cross-platform compatibility on Windows, Linux and macOS.

As for the GUI implementation process, the initial code was generated by querying ChatGPT, which provided some skeleton code with basic functions. The initial query was:

*Please generate a Python wxPython SDI application. The program starts in graph editing mode. Nodes of the graph are entered first. Nodes are entered by clicking the LEFT button of the mouse. Node data to be entered includes node name and node type (and node, or node). Then, nodes are connected with links. Links between nodes are entered by first clicking a LEFT button over a starting node, and then clicking LEFT button over an ending node. Link data includes optional negation (~) label. Node data is*

*edited by double clicking the LEFT button of the mouse over the node. Links between nodes are edited by first clicking LEFT button over a starting node, and then clicking LEFT button over an ending node of an existing link. Nodes are moved by pressing the LEFT button of the mouse over node, dragging it to the new position and then depressing a LEFT button of the mouse. Elements of a graph are deleted by selecting it for editing and choosing Delete button on opened screen form.*

The generated code, not presented here due to size, contained two errors: event handlers were not attached to the corresponding component but to the parent window, and a certain division of two integers was not converted to an integer as expected, so the function call with that expression as a parameter was not recognized. Some required functionality was not implemented, such as creating branches or moving nodes by dragging the mouse. There is also an aesthetic problem that branch ends are displayed over the nodes.

The interaction with ChatGPT continued with the following: *Please add creation of links between nodes according to previously described interface, which was implemented correctly, by adding another 17 lines of code*. The resulting code is shown in Figure 5. The next query to ChatGPT was: *Please add to this code a functionality of moving node by pressing the LEFT button of the mouse over node, dragging it to the new position and then depressing a LEFT button of the mouse*.

This resulted in another 12 lines of code being added. The functionality to move graph nodes by dragging them with a mouse was correctly implemented, but the functionality of adding branches to a graph was removed by mistake. The problem was that the mouse event handler on_left_down was not updated properly to cover all situations.

Since the maximum to which the GUI can be generated with the help of ChatGPT was reached, the rest of the implementation was mostly manual (see Table 3, Project A row for statistics on code).

By looking at the structure of the Python code implementing the Myers algorithm originated from decompilation of the original code, it is evident that the code was created from C code, therefore no structures that would naturally correspond to Python were used. For example, for CEG graph representation, several arrays of different objects (originating from C structures) are used, depending on the type of nodes and branches of the graph. The algorithm also uses dynamic structures based on linked lists.

A dilemma arose: whether to use those existing structures in the GUI part when implementing the functionality of creating and editing graphs or to use different structures natural to Python. It was decided in favor of the second option, so that the graph is represented by two dictionaries, for nodes and for branches of the graph. In Python they are declared in two lines compared to as many as 70 lines for the same graph representation in the decompiled C code.

```python
import wx
class Node:
  def __init__(self,name,node_type,x,y):
    self.name = name
    self.node_type = node_type
    self.x = x
    self.y = y

class Link:
  def __init__(self,start_node,end_node,\
  negation=False):
    self.start_node = start_node
    self.end_node = end_node
    self.negation = negation

class GraphFrame(wx.Frame):
  def __init__(self, parent, title):
    super(GraphFrame, self).__init__(parent,
title=title, size=(600, 400))

    self.panel = wx.Panel(self)
    self.panel.Bind(wx.EVT_PAINT, self.on_paint)
    self.panel.Bind(wx.EVT_LEFT_DOWN, self.on_left_down)
    self.panel.Bind(wx.EVT_LEFT_DCLICK,
        self.on_left_double_click)
    self.panel.Bind(wx.EVT_RIGHT_DOWN,
        self.on_right_down)
    self.panel.Bind(wx.EVT_KEY_DOWN, self.on_key_down)

    self.nodes = []
    self.links = []
    self.selected_node = None
    self.start_node = None
    self.end_node = None

  def on_paint(self, event):
    dc = wx.PaintDC(self.panel)
    dc.Clear()

    for node in self.nodes:
      dc.DrawCircle(node.x, node.y, 20)
      dc.DrawText(node.name, node.x-10,\
          node.y - 10)

    for link in self.links:
      dc.DrawLine(link.start_node.x,\
       link.start_node.y, link.end_node.x,\
       link.end_node.y)
      if link.negation:

        dc.DrawText("~",\ int((link.start_node.x +\
link.end_node.x) / 2),\ int((link.start_node.y +\
         link.end_node.y) / 2))

  def on_left_down(self, event):
    pos = event.GetPosition()
    node = self.get_node_at(pos)
    if node is None:
      name = wx.GetTextFromUser(\
         "Enter node name:", "Node Name")
      if name:
        dialog = wx.SingleChoiceDialog(\
          self, "Select node type:",\
          "Node Type", ["Node", "Node"])
        if dialog.ShowModal() == wx.ID_OK:
          node_type =\ dialog.GetStringSelection()
          self.nodes.append(Node(name, node_type, pos.x,
          pos.y))
          self.Refresh()
        dialog.Destroy()
    else:
      if self.start_node is None:
        self.start_node = node
      else:
        self.end_node = node
        self.add_link()
        self.start_node = None
        self.end_node = None

  def on_right_down(self, event):
    pos = event.GetPosition()
    node = self.get_node_at(pos)
    if node:
      self.selected_node = node
      menu = wx.Menu()
      delete_item =\ menu.Append(wx.ID_ANY, "Delete")
      self.Bind(wx.EVT_MENU,\ self.on_delete,
       delete_item)
      self.PopupMenu(menu, pos)
      menu.Destroy()

  def on_delete(self, event):
    if self.selected_node:
      self.nodes.remove(self.selected_node)
      self.links = [link for link in self.links if
link.start_node != self.selected_node and link.end_node
!= self.selected_node]
      self.selected_node = None
      self.Refresh()

  def on_left_double_click(self, event):
    pos = event.GetPosition()
    node = self.get_node_at(pos)
    if node:
      name = wx.GetTextFromUser(\
        "Enter node name:", "Node Name",\
        default_value=node.name)
      if name:
        dialog = wx.SingleChoiceDialog(\
          self, "Select node type:",\
          "Node Type", ["Node", "Node"])
        if dialog.ShowModal() == wx.ID_OK:
          node_type = dialog.GetStringSelection()
          node.name = name
          node.node_type = node_type
          self.Refresh()
        dialog.Destroy()

  def on_key_down(self, event):
    if event.GetKeyCode() == wx.WXK_DELETE:
      self.on_delete(event)

  def get_node_at(self, pos):
    for node in self.nodes:
      if (pos.x - node.x) ** 2 \
      + (pos.y - node.y) ** 2 <= 20 ** 2:
        return node
    return None

  def add_link(self):
    negation = wx.YES_NO | wx.CANCEL | wx.YES_DEFAULT |
      wx.ICON_INFORMATION
    dialog = wx.MessageDialog(self, \
      "Do you want to add negation to the\ link?",
      "Negation", negation)
    result = dialog.ShowModal()
    if result == wx.ID_YES:
      self.links.append(\
        Link(self.start_node,\ self.end_node,
        negation=True))
    elif result == wx.ID_NO:
      self.links.append(\
        Link(self.start_node,\ self.end_node))
    dialog.Destroy()
    self.Refresh()

class GraphApp(wx.App):
  def OnInit(self):
    frame=GraphFrame(None,"Graph Editor")
    frame.Show(True)
    self.SetTopWindow(frame)
    return True

if __name__ == '__main__':
  app = GraphApp()
  app.MainLoop()
```

**Fig. 5. Python GUI code that resulted from querying ChatGPT.**

This produces a new dilemma: should the Python implementation of Myers' algorithm be restructured to use the new structures or not? In the second case, data marshalling must be done from the GUI part of the application to the part that implements the Myers algorithm. It was decided in favor of this second approach since Myers' algorithm worked well with the structures inherited from C, and the level of changes required to change the structures would be large and would nullify the gains of transpiling the code from the existing C code. That is why a special function was implemented that converts the new graph representation into the one needed by the Myers algorithm. The final GUI is shown in Figure 6.

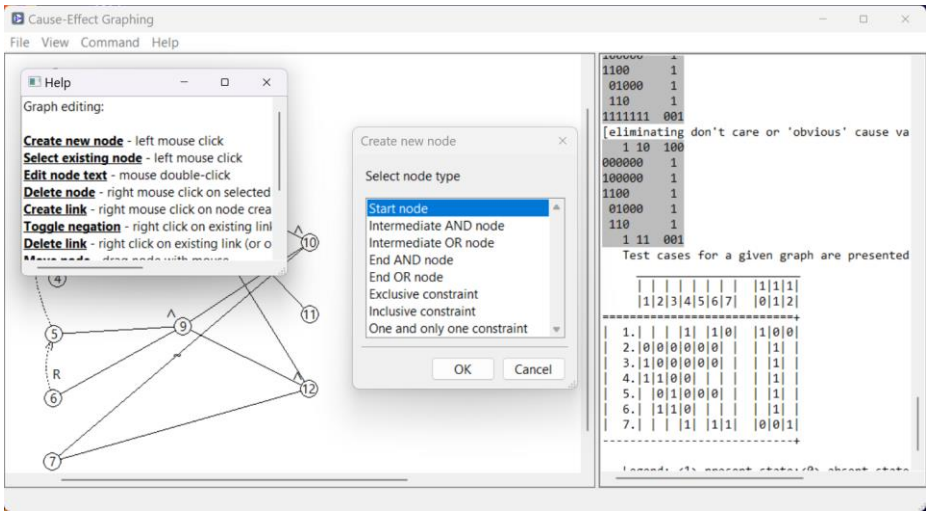



**Fig. 6. The new application's GUI.**

During black box testing of the new application and repeating the tests in the original application, a couple of previously undiscovered bugs in the original application were discovered, primarily related to the GUI, while testing both Myers' algorithm implementations left the impression that the code quality is good, i.e., that this scavenging process preserves the quality of the original implementation.

## 5. Experimental results

For investigation, we have selected three projects that share many common characteristics. Table 1 provides an overview of the basic characteristics. Project A is specific in that the source code was not available. The goal of the study was to extract some interesting subsets of each project, and to translate that part to working a Python application utilizing wxPython GUI library. Python and especially the wxPython library has somewhat similar structure and semantics to the original C++ libraries. Modernization enables further evolution of programs in popular languages and libraries that are under active development, as well as running programs on all popular desktop platforms: Windows, Linux, and macOS. For Project A, we decided to create a modified

user interface, and for the other projects, the interface should be identical to the original one.

Table 2 shows the relative sizes of the original project code according to two classifications:

- functional and object-oriented;
- to code that implements algorithms and uses memory structures, code that deals with files and databases, and code that deals with graphics input-output.

The third classification is between the decompiled code (Project A) and original C/C++ code (other projects). In this way, we can independently analyze the performance of LLM translation for different types of code. The unit for these classifications was a single function (despite the fact that sometimes different classes of code are interleaved even within the same function), and the compilation units were split accordingly.

Table 1: Basic information about selected projects

| | Project name | Year of software development, Size [in LOC], Effort | Programming language / technology | Runtime environment | Source code available | Documentation and test cases | Analyzed part of the software |
|---|---|---|---|---|---|---|---|
| **Project A** | Software testing tool | 1995, ~6500 LOC, 0.5 man/years | C, Windows 16bit API | Windows 2000 | Binary code with debug info available | User and technical docs, manual tests available | Myers alg 1818 LOC (27.6%) decompiled |
| **Project B** | Physics laboratory software [20] | 2001, ~44000 LOC, 3 man/years | C++, Windows 16bit API, custom classes | Windows XP | Yes, build environment Borland C++ 5.02 | User and technical docs, tests available on simulated hardware | 12 272 LOC (27.9%) subsystem areascan |
| **Project C** | Maintaining database of knowledge quizzes and test assembly [21] [22] | 2010, ~62300 LOC, 4.5 man/years | C++, MFC GUI library, custom ORM classes | Windows 7 | Yes, build environment Visual C++ 2009 | User documentation, test data | 12 876 LOC (20.6%), subsystem test assembly |

Table 2: Characteristics of analyzed source code (in percentages)

| | Functional / Object Oriented Code | Memory structures and algorithms | Working with files / databases | Graphic input / output |
|---|---|---|---|---|
| **Project A** | 100% / 0% | 79.10% | 20.90% | 0% |
| **Project B** | 10.1% / 89.9% | 25.65% | 12.40% | 61.95% |
| **Project C** | 4.9% / 95.1% | 26.50% | 46.20% | 27.30% |

Table 3 gives the characteristics of the new code in percentages: how much code was obtained by translation using ChatGPT and used without modification, how much ChatGPT code had to be manually modified/supplemented, how much code was obtained by generating in ChatGPT from the textual description, and how much such code had to be manually modified/supplemented. Two types of trivial manual modifications of

generated code were not taken into account: adding import declarations and renaming variables/methods. Sometimes, ChatGPT generated code with intact identifiers, e.g., OnLoadParameters from C++ remains the same, but sometimes it imposed Python naming conventions, e.g., on_load_parameters.

We believe that this integrated way of measuring code translation performance using LLM better reflects the real scenario of legacy application modernization than isolated measurement on individual functions, since real projects most often do not have an extensive set of unit tests that are a prerequisite for this second method of measurement.

The translated code from Project A needed more manual correction effort than code from other two projects (B and C), probably due to the fact that Project A code was decompiled from binaries. The accuracy rate of over 88% for the machine-translated code for projects B and C seems very satisfactory and is generally higher than in other studies. This is likely due to the carefully crafted ChatGPT prompts and specifying target libraries, as explained in the methodology chapter. On the other hand, the code generated based on the textual description of the user interface in Project A suffered a modification level of over 93% (Table 4), which raises the question of how useful ChatGPT was at all in that activity (only 2.02% of the code thus generated remained unchanged in the final code of the project).

Table 3: Characteristics of the final code

| | Automatically generated | | Manual changes/additions | |
|---|---|---|---|---|
| | **Translation** | **Generated from chat** | **On translated code** | **On the code from the chat** |
| **Project A** | 52.12% | 2.02% | 16.67% | 29.19% |
| **Project B** | 88.43% | 0.00% | 11.57% | 0.00% |
| **Project C** | 88.21% | 0.00% | 11.79% | 0.00% |

We classified the final code in the same way as the initial code and Table 4 shows the performance of the LLM translation according to the types of code. Projects B and C give similar results, while Project A is specific due to the use of decompiled code and the generation of a GUI from a textual description. As we said before, on decompiled code LLM gives lower performance, probably because its structure does not look like the structure of human written code ChatGPT was trained on. It is also noticeable that the performance is weaker on the OO code compared to the functional one. OO code is more complex and sometimes has deep inheritance hierarchies, C++ has language constructs that Python does not directly support, such as templates and function overloading, and ChatGPT does not always get right semantics for each C++ standard library operations.

| Original C++ code (a fragment) | Code generated by ChatGPT (a fragment) |
|---|---|
| // ok if called with an empty array p_niz<br>template <class C><br>BOOL MenadzerBaze::Get(CArray<C,C&> &p_niz, const db_id_t p_fkId /* = 0 */, LPCTSTR p_fkName /* = NULL */, BOOL p_isInternal /* = FALSE */) {<br>C c; | # throws error if called with an empty array p_niz<br>class MenadzerBaze:<br>def get(self, p_niz, fk_id=0, fk_name=None, is_internal=False):<br>obj = p_niz[0].__class__() # Create an instance of the object class |

| Original C++ code (a fragment) | Code generated by ChatGPT (a fragment) |
|---|---|
| std::list<stanje_t> stanja;<br>std::vector<stanje_t> novaStanja;<br>// inserts novaStanja in the original order at the beginning of stanja<br>stanja.insert(stanja.begin(), novaStanja.begin(), novaStanja.end()); | stanja = deque()<br>novaStanja = []<br># erroneous ChatGPT generated code<br># inserts elements of novaStanja in reverse order<br>stanja.extendleft(novaStanja) |

As in other studies, lower performance was obtained for code working with data files (or a custom object-relational mapping as in Project B [20]) than for other types of code. Translating GUI code is not a big problem for ChatGPT because such code has regular patterns, even in situations (Project B) where it translates functional C code (Windows API) to object-oriented GUI code (wxPython).

Table 4: Percentage of manual changes in the final code by type of source code

| | Project A | Project B | Project C |
|---|---|---|---|
| **Functional Code** | 26.90% | 7.77% | 7.39% |
| **Object Oriented Code** | -- | 13.29% | 12.24% |
| **Memory structures** | 26.50% | 6.62% | 10.26% |
| **Working with files** | 28.20% | 32.16% | 14.94% |
| **Graphic input / output** | 93.50% | 11.15% | 7.73% |

Table 5 gives a rough estimate of how much effort was saved by using LLMs in these projects. The assessment is based on the basic COCOMO formula in organic mode [23], i.e., for small projects and experienced team:

$$\text{effort in man/months} = 2.4\ (\text{size in KLOC})^{1.05}.$$

The saving is expressed as a percentage difference between the value of effort if the entire code would be translated manually without ChatGPT and the value of effort to correct the code generated by ChatGPT, based on the size of the new code.

Table 5: Estimated savings in code translation

| | Project A | Project B | Project C |
|---|---|---|---|
| **Final code size [in KLOC]** | 2028 KLOC | 3285 KLOC | 5463 KLOC |
| **Effort1 (full manual translation)** | 5.04 | 8.37 | 14.27 |
| **Effort2 (changes to ChatGPT code)** | 2.22 | 0.87 | 1.51 |
| **Effort savings (%)** | 55.9% | 89.6% | 89.4% |

***Threats to validity***

Our experience shows that translation using an LLM is very sensitive to the source and target languages of the translation and their mutual semantic distance, to the libraries used, to the querying method, and to the size of the classes and functions being translated. A small change in any of these factors could potentially yield significantly different results. In this sense, we believe that this study shed more light on the issue of modernization of real applications than studies that take a wider range of language pairs and isolated program fragments. However, the study was limited to one specific class of

desktop applications and to specific technology from a specific time period. In this sense, a similar study should be repeated in other classes of applications and technologies.

## 6. Conclusion

This paper explores the modernization of legacy C++ applications from the 1990s and early 2000s to Python as an example of a modern language and platform. Advances in translation technology from one programming language to another, particularly the use of LLMs, have made this task cost-effective. A methodology has been defined, and an empirical study has been conducted on several selected software projects.

The contributions of this research, compared to existing studies on translation performance using LLMs, included the use of real-world projects and an integrated approach to code translation, specifically for language libraries. Scenarios that appear in practice have also been taken into account, including translation of decompiled code in the event that the source code of the legacy application is not available. Two classifications of legacy code and translation performance measured against the defined classes have been developed.

The findings of this study could benefit other types of legacy system modernization, such as transforming monolithic web applications into microservices, and contribute to the further improvement of LLMs and the communication methods used with them.

## Acknowledgments

This research was supported by the Science Fund of the Republic of Serbia, Grant No 11113, *Software for Text Offences Prevention in Serbian: Al-driven Hate Speech Detection* - STOP.

## References

[1] D. L. Parnas, Software aging, in Proceedings of 16th International Conference on Software Engineering, IEEE, May 1994, pp. 279-287.

[2] -, The Internet Archive Software Collection, available at: https://archive.org/details/software (accessed 10 October 2023).

[3] Rocket UniVerse Datasheet, 2019, available at: https://www.rocketsoftware.com/sites/default/files/resource_files/rs_ds_mv_universe_mar2019_Oct2019_finaldigital.pdf (accessed 17 December 2023).

[4] M. Razavian, Knowledge-driven migration to services, PhD Thesis, Vrije Universiteit Amsterdam, 2013.

[5] T. Claburn, Upset Equation Editor was killed off? Now you can tell Microsoft to go forth and multiply: App back from the dead, The Register, available at: https://www.theregister.com/2018/01/16/microsoft_equation_editor_patched/ (accessed 17 May 2024).

[6] X. Chen, C. Liu, and D. Song, Tree-to-tree neural networks for program translation, Advances in neural information processing systems, vol. 31, 2018.

[7] M. Szafraniec, B. Roziere, H.J. Leather, P. Labatut, F. Charton, and G. Synnaeve, Code Translation with Compiler Representations, In the 11th International Conference on Learning Representations, 2022.

[8] C2Rust Transpiler: Migrate C code to Rust, available at: https://github.com/immunant/c2rust (accessed 08 February 2025).

[9] H. Zhong, S. Thummalapenta, T. Xie, L. Zhang, and Q. Wang, Mining API Mapping for Language Migration. In Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1 (Cape Town, South Africa) (ICSE '10), Association for Computing Machinery, New York, NY, USA, 2010, pp. 195–204, doi: 10.1145/1806799.1806831

[10] M. Lachaux, B. Roziere, L. Chanussot, and G. Lample, Unsupervised translation of programming languages, *arXiv preprint arXiv:2006.03511*, 2020.

[11] B. Roziere, J. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample, Leveraging Automated Unit Tests for Unsupervised Code Translation, In International Conference on Learning Representations, 2021.

[12] Z. Yang, F. Liu, Z. Yu, J.W. Keung, J. Li, S. Liu, Y. Hong, X. Ma, Z. Jin, and G. Li, Exploring and unleashing the power of large language models in automated code translation, *Proceedings of the ACM on Software Engineering*, 2024, 1.FSE: 1585-1608.

[13] B. Roziere, M. Lachaux, L. Chanussot, and G. Lample, Unsupervised translation of programming languages, Advances in Neural Information Processing Systems 33, 2020, 20601–20611.

[14] D. Velasevic, D. Bojic, and D. Novicic, TESTGRAPH: software tool for automatic generation of test case patterns, Info Science, 5(2), 1997, pp. 4-20.

[15] G. Myers, C. Sandler, T. Badgett, The Art of Sofware Testing, 3rd edition, 2012, John Wiley & Sons.

[16] ISO/IEC/IEEE 29119-4:2021(en) Software and systems engineering – Software testing – Part 4: Test techniques.

[17] E. Krupalija, E. Cogo, Š. Bećirović, I. Prazina and I. Bešić, Cause-effect Graphing Technique: A Survey of Available Approaches and Algorithms, *2022 IEEE/ACIS 23rd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing* (SNPD), Taichung, Taiwan, 2022, pp. 162-167, doi: 10.1109/SNPD54884.2022.10051799.

[18] W.S. Jang, and R.Y.C. Kim, Automatic Cause–Effect Graph Tool with Informal Korean Requirement Specifications, Applied Sciences 12, no. 18: 9310, 2022, doi: 10.3390/app12189310.

[19] The Ghidra decompiler, 2022, available at: https://ghidra-sre.org/ (accessed 11 November 2023).

[20] XCTL-Projekt: Software-Sanierung, Dept. of Computer Science, Humboldt-Universität zu Berlin, May 2002, available at: https://www2.informatik.hu-berlin.de/swt/projekt98/index.html (accessed 16 January 2025).

[21] D. Bojic, A. Bosnjakovic, J. Protic, and I. Tartalja, A Modified Hill-Climbing Algorithm for Knowledge Test Assembly Based on Classified Criteria, International Journal of Software Engineering and Knowledge Engineering, vol. 26, no. 06, 2016, pp. 953-980, doi: 10.1142/S0218194016500327

[22] M. Ignjatovic, D. Bojic, B. Furlan, and I. Tartalja, Potential of knowledge discovery in automated test assembly, Telfor Journal, vol. 7, no. 2, 2015, doi: 10.5937/telfor1502108I

[23] B. Boehm, R. Valerdi, J. Lane, and A.W. Brown, COCOMO suite methodology and evolution, *CrossTalk The Journal of Defense Software Engineering*, *18*(4), 2005, pp.20-25.